

# Theoretical Documentation (RA7 Report)

Title: RA7 – Web Services (API Development)

Cristian Alvarez / Server-Side Web Development

Fecha 12/02/2026

## What is a Web Service?

A web service is a system that allows communication between applications over the internet using standard protocols like HTTP. It enables data exchange between different systems regardless of programming language or platform.

---

## What is a REST API?

A REST API is an interface that follows **REST architecture principles**, allowing applications to interact with resources using standard HTTP methods like **GET**, **POST**, **PUT**, and **DELETE**. Data is typically exchanged in **JSON format**.

---

## Common Use Cases of REST APIs

- Mobile applications connecting to a backend
  - Web frontends retrieving dynamic data
  - Third-party integrations
  - Microservices communication
- 

## Advantages of Using an API as an Access Layer

- **Separation of business logic** from UI
- **Scalability** and easy maintenance
- **Reusability** for multiple clients

- **Interoperability** between different systems
  - **Security**: control access to data
- 

## REST Architecture Principles

### Stateless Communication

Each request contains all necessary information; the server does not store client session.

### Resource-Based URLs

Resources are identified via URLs, for example:

```
/api/tasks  
/api/tasks/1
```

### HTTP Methods

- **GET** → retrieve resources
- **POST** → create new resources
- **PUT/PATCH** → update resources
- **DELETE** → remove resources

### HTTP Status Codes

- **200** → success
  - **201** → created
  - **404** → not found
  - **422** → validation error
- 

## Technologies Used

- **HTTP** → communication protocol
- **JSON** → data exchange format
- **REST** → API architecture style
- **Laravel 12** → backend framework
- **Postman** → API testing tool

## API Design Documentation

### Base URL

`http://127.0.0.1:8000/api`

### Endpoints

| Method | Endpoint    | Body / Params                                                                  | Response Example                                                                                     | Status Codes  |
|--------|-------------|--------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|---------------|
| GET    | /tasks      | —                                                                              | <code>{ "id":1, "title":"Mi primera tarea", "description":"Probando API", "completed":false }</code> | 200           |
| POST   | /tasks      | <code>{ "title":"string", "description":"string", "completed":boolean }</code> | <code>{ "id":1, "title":"Mi primera tarea", "description":"Probando API", "completed":false }</code> | 201, 422      |
| GET    | /tasks/{id} | —                                                                              | <code>{ "id":1, "title":"Mi primera tarea", "description":"Probando API", "completed":false }</code> | 200, 404      |
| PUT    | /tasks/{id} | <code>{ "title":"string", "description":"string", "completed":boolean }</code> | <code>{ "id":1, "title":"Updated Task", "description":"Updated", "completed":true }</code>           | 200, 422, 404 |
| DELETE | /tasks/{id} | —                                                                              | <code>{ "message":"Task deleted successfully" }</code>                                               | 200, 404      |

### Authentication

Not implemented

For this RA7 practice → API is public

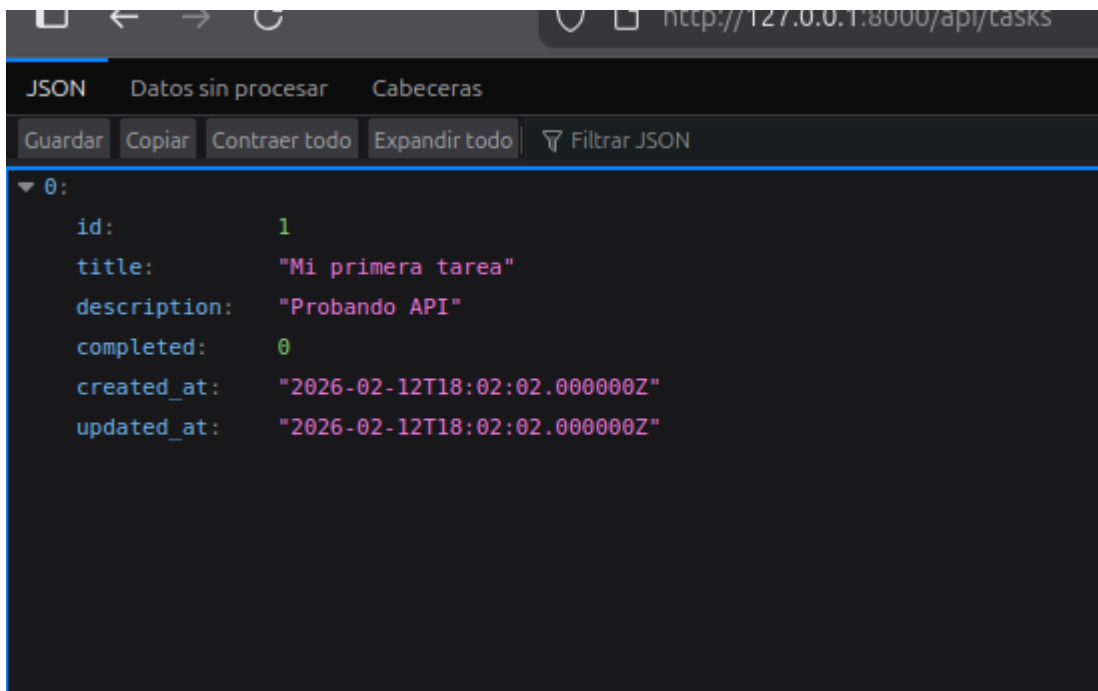
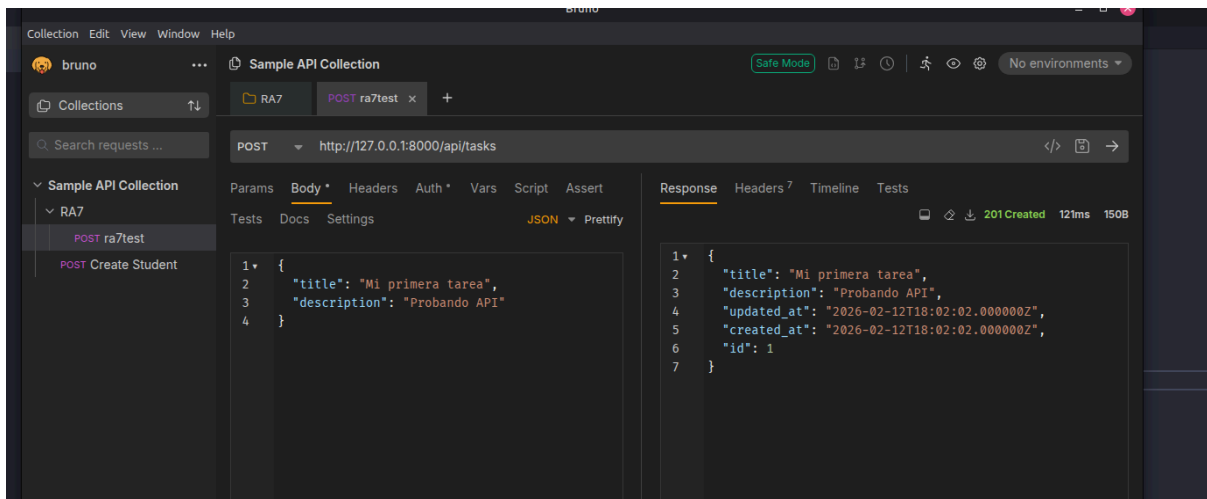
### API Source Code Summary

`routes/api.php` → routes registered

`app/Http/Controllers/TaskController.php` → CRUD logic

`app/Models/Task.php` → fillable fields: title, description, completed

`database/migrations/...create_tasks_table.php` → table with id, title, description, completed, timestamps



GET /api/tasks → initially returns []

POST /api/tasks → create a task, check 201 Created

GET /api/tasks → confirm task exists

GET /api/tasks/{id} → test existing and non-existing task (404)

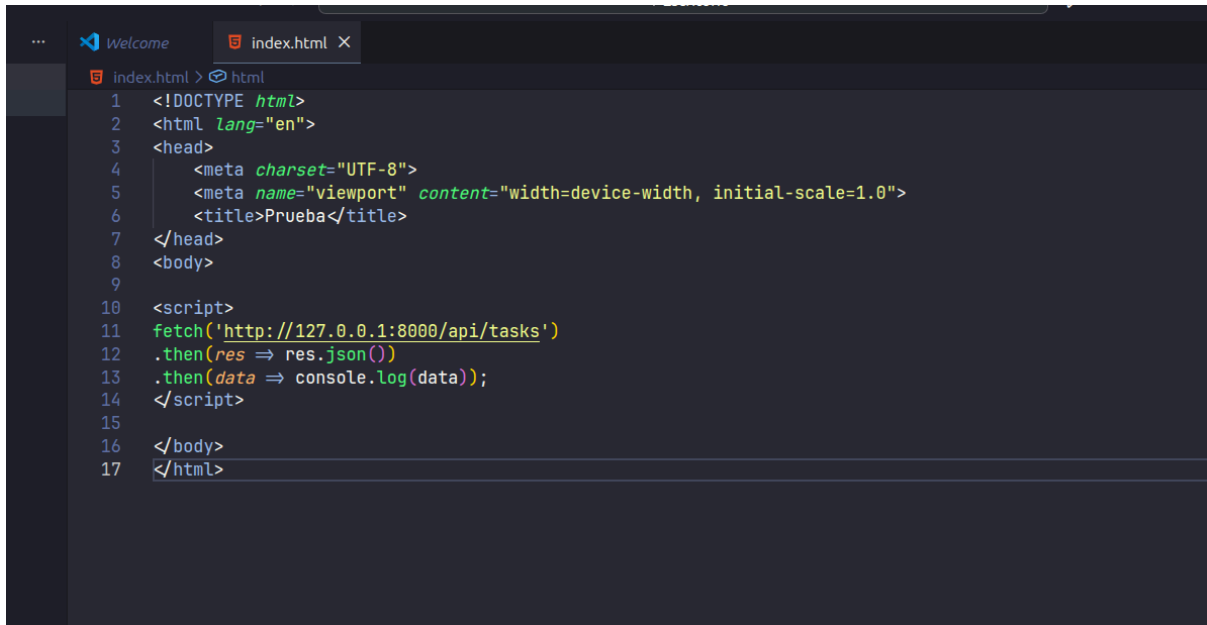
PUT /api/tasks/{id} → update task, check updated JSON

DELETE /api/tasks/{id} → delete task, check success message

API Consumption Evidence

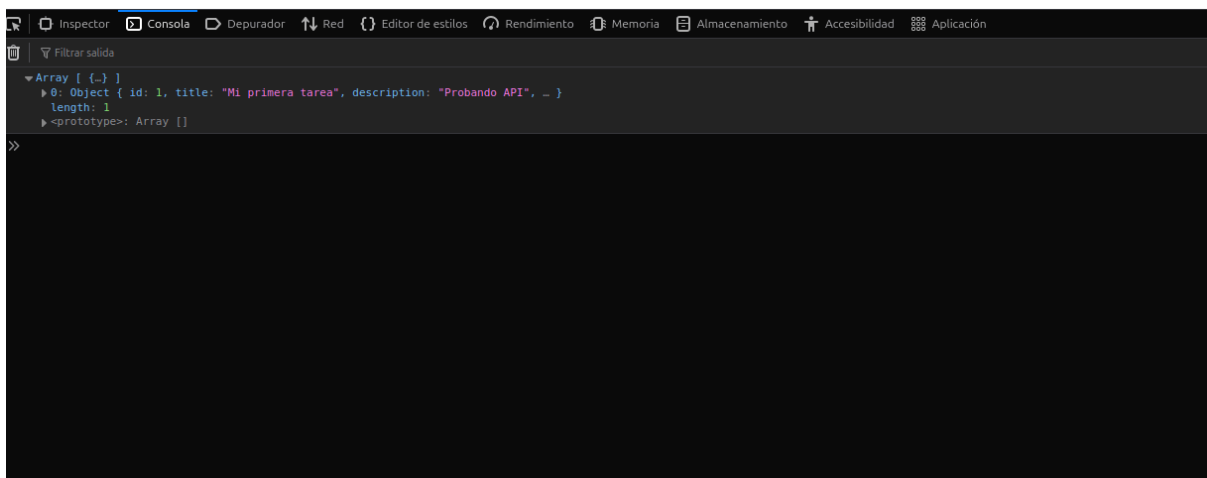
Example JS fetch (already suggested):

```
<script>
fetch('http://127.0.0.1:8000/api/tasks')
.then(res => res.json())
.then(data => console.log(data));
</script>
```



The screenshot shows a code editor with a file named 'index.html'. The code is as follows:

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Prueba</title>
7 </head>
8 <body>
9
10 <script>
11   fetch('http://127.0.0.1:8000/api/tasks')
12   .then(res => res.json())
13   .then(data => console.log(data));
14 </script>
15
16 </body>
17 </html>
```



The screenshot shows the browser's developer console with the following output:

```
>>
Array [ { } ]
  0: Object { id: 1, title: "Mi primera tarea", description: "Probando API", ... }
  length: 1
  <prototype>: Array []
```